The life of a sample: from `eb18` to `eb22g`, traced

Status: working paper, originally traced 2026-10-02 on `eb18`, updated **2026-11-05** for **`eb22g`** (`PAPER_3_DOCUMENT_EVENTS_AND_COREF.md`).

Status update 2026-11-05: While the high-level pipeline skeleton (`JSONL -> label map -> sliding window -> argument coref -> negative pools -> collator -> boundary & record heads -> loss -> AdamW`) is the same, **eight load-bearing stages of the sample lifecycle changed between `eb18` and `eb22g`**: 1. **Stage `S2.5` (Argument Coreference & Multi-Occurrence Grouping):** `expand_argument_coreferents` (`gliner2/processing/argument_coref.py`) attaches coreferent entity mentions from `output.coreferences` with `_coref_group` tags, and `_build_outputs` (`processor.py`) + `_build_record_field_target` (`boundary_preprocessing.py`) compile coreferent mentions and repeated string occurrences as **span alternatives inside a single `RecordFieldTarget.values` entry** (`len(ft.values) = 1`) rather than separate required fillers. 2. **Stage `S3` (`ExtractorDataset.__getitem__`):** `remove_events_prob` is now `0.0` (eliminating the double 20% event-schema dropout on top of negative sampling), and `NegativePoolSampler` (`negative_pools: auto`, `negative_labels_per_dim: {entities: 20, events: 20}`, `negative_gold_full_roles: true`) injects up to 20 absent entity labels and 20 absent event types per document while listing all unargued roles on gold event types (`labels/unified-full-v4.yaml`). 3. **Stage `S5` (Span Scoring & Residual Init):** `enable_rotary_endpoints: false` and `span_residual_zero_init: true` zero-initialize the post-`eb21` span residual terms (`endpoint`, `content_bias`, `attn_out`, with `inside_bias = 1`) so span logits start at the clean `eb21` pair logit; under `candidate_pool: per_query`, the 70 unused `shared_pool_*` parameters are frozen (`requires_grad = False`). 4. **§2.3–2.4 (`candidate_pair_loss` & Absent-Query Hard Negatives):** In `eb21`–`eb22f`, `hard_negative_keep_all_when_absent: true` kept all 768 candidates on every absent query (`~85–90%` of queries under `k = 20` negative sampling) in one global denominator, diluting positive `pair_loss` by 7,700× (`pair_loss ~ 0.005–0.011`). In **`eb22g`**, `hard_negative_keep_all_when_absent: false` (absent queries keep `minimum_hard_negatives: 16`) and **`pair_split_reduction: true`** average positive BCE over `pos_mask.sum()` and negative BCE over `neg_mask.sum()` (`0.5 * (pos_mean + neg_mean)`), restoring `pair_loss` to `~1.0–1.9` and `boundary_head.pair_scorer` to the 4th largest gradient norm in the model. 5. **§2.10 (`record_field_loss` in `natural` mode):** - **Anchor self-row dropped (`record_drop_anchor_field_loss: true`):** `field_idx = 0` (the trigger anchor that seeded the instance) is excluded from the field loss (`1 anchor rows dropped`), so `record_field_loss` averages only over argument roles `f >= 1`. - **Split positive/hard-negative BCE mean (`_list_field_bce`):** Instead of averaging over all 768 candidates (`eb18`) or averaging `G` positives and `K = 8` hard negatives in one `(G + K)` denominator (`eb21`–`eb22e`, equilibrium `P = 1/9 = 0.111`), `_list_field_bce` (`record_role_hard_negatives: 8`) computes `pos_loss.mean() + neg_loss.mean()`, with candidates overlapping unresolved gold spans masked out on partial matches (`_unresolved_overlap_cols`). - **Unresolved gold roles withheld:** When a non-empty gold role has no candidate in the pool (`gold_cols = []`), it is **withheld (`0.0` loss, excluded from `fields_counted`)** rather than trained toward `0.0`. 6. **§2.10b (Relational Junction Layer & Contrastive Column Loss):** `record_link_mode: junction` adds the role-gated bilinear + relative-geometry junction term to `_assign_logits`, and

`record_link_column_weight: 0.3` adds a per-column listwise cross-entropy over triggers for each gold argument column (`_column_loss`). 7. **§2.10c (Multi-Mention Coreferent Ownership, `TriggerLink`, and `EntityCorefLink`):** `record_coreferent_ownership: true` seeds a gold instance row for every coreferent trigger mention in `values[0]` (normalized by `1 / |G|`); `record_coref_link: true` (`record_coref_link_weight: 0.3`, `coref_link_distance: mlp`, width 64) trains pairwise trigger coreference; and `entity_coref_link: true` (`entity_coref_link_weight: 0.3`) trains pairwise entity-mention coreference (`entity_coref_loss`). 8. **§2.12 (Silent Exception Guards Removed):** `ValueError` and `IndexError` in record and relation losses are no longer silently swallowed; only `TargetCapacityError` is caught and handled per `on_capacity_exceeded: truncate_with_warning`.

Purpose

This follows ONE real training record from its JSONL line to the optimizer step. At each stage it gives the real values the pipeline held, and the loss equation exactly as the code implements it. The documented flow and the actual flow drift apart. This paper is the place to compare them, and four corrections below came from doing exactly that.

Evidence grades used throughout: - **one record** / **one batch**: a single traced example. It shows the mechanism, not a rate. - **measured**: a sample-based rate with its n given.

Reproduce

```
uv run python tools/train/trace_sample_life.py --corpus casie            # S0–S7 on one real record
uv run python tools/train/measure_gradient_shares.py --batches 8 [--absent-reduction separate]
uv run python tools/train/measure_absent_dilution.py                     # pi_P over the real mix
uv run python tools/data/adjudicate_absent_events.py --per-corpus 50     # is "absent" true?
```

`trace_sample_life.py` has two self-checks: - **S6 gate:** the weighted sum of the logged terms must reproduce the model's own loss. - **S7 gate:** the gradient shares must sum to 1.

---

1. Trace diagram (eb18-balanced, one CASIE record)

Real values come from `trace_sample_life.py`, seed 0, eb18 checkpoint, CPU, train mode.

```
S0  RAW JSONL  data/casie.train.jsonl
    575 chars: "The boss of a company held to ransom by computer hackers ..."
    gold: entities Person x5, Organization x2, Capabilities, PaymentMethod, Money
          events   Cyber.Ransom x3  (triggers 'held to ransom' | 'demanded money' | 'demanded')
        |
S1  LABEL TRANSFORM   labels_file: labels/unified-full-v4.yaml     (no change for this record)
        |
S2  SLIDING WINDOW    max_len 4096 subwords, stride 3072  ->  1 chunk (114 subword tokens)
        |
S2.5 ARGUMENT COREF   expand_argument_coreferents (training.expand_argument_coref: true)
    - attaches coreferent entity mentions from output.coreferences with _coref_group tags
      so coreferent mentions compile as span alternatives of one filler (len(ft.values) = 1)
        |
S3  ExtractorDataset.__getitem__   (runs in a worker; re-samples on EVERY call)
    - [eb18]: remove_events_prob 0.2 could drop a gold event type; +1 entity, +1 event negative
    - [eb22g]: remove_events_prob 0.0 (no gold event types dropped);
      NegativePoolSampler (negative_pools: auto, k=20 entities, k=20 events,
      negative_gold_full_roles: true) injects up to 20 absent entity labels + 20 absent
      event types and lists all unargued roles on gold event types
        |
S4  COLLATOR    107 words, 218 subword ids, queries expanded per schema
    - an EVENT TYPE expands to a trigger query + one query per role:
      absent event types -> trigger + all roles, all ABSENT (no gold)
      present event types -> trigger + all roles (gold spans grouped by filler alternatives)
    - mention_mask [B,Q,G] = gold spans per query; record_specs: natural groups
      (event_records: true -> anchor = trigger field)
        |
S5  FORWARD (train mode)
    encoder mmBERT-base, FA2, bf16 on GPU
      -> boundary marginals: start/end logits [B,Q,L+1], inside logits [B,Q,T]
      -> proposal: top-128 starts x conditional ends, per query (candidate_pool: per_query)
         + GOLD INJECTION with prob p_inj(t): 1.0 for the first 15% of planned optimizer
           steps, then linear to 0.25 (proposal_gold: identity)
         -> up to 768 candidates per query (training_candidate_budget)
      -> pair scorer (compat + start marginal + end marginal + zero-init span residuals),
         null head, count head, entity_coref_link head
      -> record head, natural mode (record_link_mode: junction):
         instances = anchor (trigger) candidates (every coreferent trigger mention in G
         seeds a gold row under record_coreferent_ownership: true);
         assign logits per field = [ABSENT column | P_ij + R_fj + bilinear_junction + geom];
         TriggerLink scores trigger x trigger pairs (64-width MLP distance)
      -> relation scorer, classifier
        |
S6  LOSSES (section 2)      model loss 0.807443
    = weighted span terms 0.801163 + record_field 0.006280 + residual -1.49e-08   GATE PASSES
        |
S7  GRADIENT   |g| = 75.96 on this record; max_grad_norm 1.0 -> clip factor 0.0132
    trainer: loss/4 per micro-batch, 4 micro-batches summed, clip_grad_norm_(1.0),
    AdamW (encoder 2e-5, task 5e-4, weight decay 0.01), cosine_restarts, warmup 0.05
```

S6 loss table (one record; head schedules at their step-0 defaults, see Finding 3)

term	value	weight	contribution	gradient norm on encoder	share of the whole-model gradient
start_loss	0.02474	1.0	0.02474	0.915	0.0071
end_loss	0.02045	1.0	0.02045	0.905	0.0087
pair_loss	0.19418	1.0	0.19418	11.65	0.1160
inside_loss	0.05584	0.5	0.02792	1.46	0.0121
soft_iou_loss	0.40840	0.2	0.08168	4.41	0.0292
rerank_listwise_loss	0.52928	0.3	0.15878	24.29	0.2920
proposal_loss	0.98217	0.3	0.29465	33.06	0.3947
consistency_loss	0.00148	0.1	0.00015	0.017	0.0001
abstention_loss	0.03061	0.2	0.00612	8.70	0.0579
count_loss	-0.03754	0.2	-0.00751	10.96	0.0807
record_field_loss	0.00628	1.0	0.00628	0.361	0.0015
record_object_loss	0.0	1.0	0	none (constant)	0
classification_loss	0.0	1.0	0	none (no classification queries here)	0

Share is defined as share_k = ⟨g_k, g⟩ / |g|², where g_k = ∇(w_k L_k). - The shares sum to 1 **by linearity, for any weights**. No normalisation of the weights is needed. - A share is a projection, not a cosine. It can be negative, or greater than 1.

---

2. Every loss term, as implemented

Notation - b = sample, q = query, ℓ = boundary position (L+1 of them), t = token, c = candidate. - BCE(x, y) = softplus(x) − x·y. - **global reduction** = Σ(elementwise·keep) / max(Σ keep, 1), pooled over the whole micro-batch (`losses.py:56–100`). - In global mode, `_reduce` **ignores** `query_mask` . - With `query_weights` , the numerator is weighted and the denominator is not. - eb18 sets no `query_weights` (`task_loss_weights` is None), so w_q = 1.

Shared masks (`model.py:782` , `:854`) - keep[b,q,ℓ] = boundary_mask[b,ℓ] ∧ query_mask[b,q] - absent[b,q,ℓ] = keep ∧ ¬ any_g mention_mask[b,q,g] (the query has no gold mention)

2.1 start / end boundary BCE (`model.py:856–873` , `losses.py:189–212`)

```
e[b,q,ℓ] = BCE(s[b,q,ℓ], y[b,q,ℓ]) · (1 if y = 1 else boundary_negative_weight)
         [boundary_negative_weight = 1.0 in eb18, 0.5 in eb22g]
```

pooled (`eb18` and `eb22g`):

```
L_start = (Σ_{keep} e) / |keep| = π_P · L̄_P + π_A · L̄_A,    π_P = N_P / (N_P + N_A)
```

separate (opt-in, `df162a0`):

```
L_start = present_loss_scale · L̄_P + absent_loss_weight · L̄_A
```

- Weight in the total: 1.0, hard-coded in `DEFAULT_LOSS_WEIGHTS` (`model.py:105`). It is not settable from YAML.
- The end loss is the same with end targets.
- **Measured** `π_P` on `eb18`'s mix (6,765 records, 20,000 simulated micro-batches of 4):
- negatives on: mean 0.713, p10 0.493, p90 0.887;
- negatives off: mean 0.862.

2.2 inside (`model.py:1019–1026`, `losses.py:470–493`)

- Balanced BCE over inside logits `[b,q,t]`.
- `keep = text_mask ∧ query_mask`; global mean.
- Weight 0.5, hard-coded.

2.3 candidate labels and hard negatives (`losses.py:251–316`, `:402–435`)

- **Label:** `z[b,q,c] = 1` iff the candidate `(s,e)` exactly equals a gold `(s,e)` of that query, i.e. by **span identity**. The candidates include injected gold.
- **Hard negatives (`select_hard_negatives`):** per query, keep all positives plus the top-`k` negatives by detached pair logit:
- In `eb18`: `k = max(5 · n_pos, 8)` (`hard_negatives_per_positive: 5`, `minimum_hard_negatives: 8`), `hard_negative_keep_all_when_absent: false`.
- In `eb21`–`eb22f`: `k = max(20 · n_pos, 16)` (`hard_negatives_per_positive: 20`, `minimum_hard_negatives: 16`), **`hard_negative_keep_all_when_absent: true`** — which kept **all 768 candidates** on every absent query!
- In **`eb22g`**: `k = max(20 · n_pos, 16)` with **`hard_negative_keep_all_when_absent: false`**, so an absent query keeps at most 16 hard negatives instead of all 768.
- The guide veto is inert (no `guide_scores`).

2.4 pair (`model.py:934–964`, `losses.py:438–493`)

Combined reduction (`eb18` through `eb22f`, `pair_split_reduction: false`):

```
L_pair = (Σ_{eff} BCE(p_bqc, z_bqc)) / |eff|,    eff = valid ∧ (z ∨ hard)
```

- Under `eb21`–`eb22f` (`hard_negative_keep_all_when_absent: true` + `k = 20` negative sampling), 85–90% of queries were absent and contributed 768 negatives each to `|eff|`, diluting positive `pair_loss` by ~7,700× (`pair_loss` pinned at `0.005–0.011`) and leaving candidate logit levels unanchored under shift-invariant listwise losses (`EB22C_SCORE_LEVEL_AUTOPSY.md`).

Split positive/negative reduction (**`eb22g`**, `pair_split_reduction: true`):

```
pos_mean = (Σ_{pos} BCE(p_bqc, 1)) / |pos|,      pos = valid ∧ (z > 0.5)
neg_mean = (Σ_{neg} BCE(p_bqc, 0)) / |neg|,      neg = valid ∧ (z ≤ 0.5) ∧ hard
L_pair   = 0.5 · (pos_mean + neg_mean)           (or pos_mean / neg_mean if only one side is non-empty)
```

- Weight 1.0, hard-coded.
- On a traced batch with 1 positive at `−5.0` and 10 absent queries × 768 candidates at `−3.0`, switching from `eb22f` to `eb22g` moves `L_pair` from `0.0492` to `5.0553` and restores the positive logit gradient from `−0.000129` to `−0.993307` (`= P − 1`).

2.5 soft-IoU (`model.py:967–982`)

```
L_iou = (Σ_{eff} BCE(p_bqc, max_g IoU(c, g))) / |eff|
```

- Same `eff` set and the same logits as pair.
- Weight `0.2 · σ_iou(t)`, with **`σ_iou(t) = max(1 − t/20000, 0)`** (`trainer.py:1417–1421`). It is off after 20,000 optimizer steps.
- **Double counting:** an exact positive gets target 1 in both pair and soft-IoU.
- **Conflict:** an overlapping hard negative gets target 0 from pair and a fractional IoU from soft-IoU, on the same logit.

2.6 rerank listwise (`losses.py:715–752` → `proposal_listwise_loss`)

```
ℓ_bq = LSE_{c ∈ valid} p_bqc − LSE_{c ∈ G_bq} p_bqc,    G_bq = {c : z > 0.5}
L_rerank = (Σ_{q : G_bq ≠ ∅} ℓ_bq) / #{q : G_bq ≠ ∅}
```

- Weight 0.3.
- A query with no gold contributes exactly 0.
- `absent_negatives_in_denominator` is False.

2.7 proposal listwise: the injected-only gold defect (`model.py:1028–1065`)

The same listwise form as 2.6, but on proposer logits (compat + start marginal + end marginal), with

```
G^prop_bq = {c : is_gold_c}    [in eb18, where is_gold marked only the INJECTED copy]
G^prop_bq = {c : z_bqc > 0.5}  [in eb21..eb22g, under proposal_gold: identity]
```

`assemble_candidates` (`proposal.py:343–450`) works like this: 1. Gold spans are injected with a ceiling score. 2. After de-duplication, only the injected copy keeps `is_gold`. 3. A gold span the proposer found **naturally**, but that this step's injection did not sample, survives as an ordinary candidate. Under `proposal_gold: injected` (`eb18`), it sat in the **denominator as a negative**, while the pair and rerank losses labelled the same span **positive**.

Probe (one real CASIE batch, `eb18`, 26 gold-bearing candidates by span identity):

p_inj	proposal-loss gold	identity-gold labelled NEGATIVE in proposal loss	proposal loss
1.0	26	0	1.6172
0.25	4	**22**	2.0605
0.0	0	26	**0.0000** (no query has gold, so the term switches itself off)

Fix (adopted in `eb21` – `eb22g`): `boundary_head.proposal_gold: identity`. It labels proposal-loss gold by span identity, the same `z` as pair and rerank.

Traced after the change on a real `eb18` CASIE batch with 16 gold candidates:

mode	p_inj	proposal-loss gold	identity-gold labelled NEGATIVE	proposal loss
injected	1.0	16	0	2.322924
injected	0.25	2	**14**	4.255371
injected	0.0	0	16	0.000000
identity	1.0	16	0	2.322924
identity	0.25	16	0	2.322924
identity	0.0	16	0	2.322924

The two modes agree to 8 decimals at `p_inj = 1.0`. Under `identity` the proposal loss no longer depends on `p_inj` for gold that was proposed naturally. Tests: `tests/models/boundary/test_proposal_gold.py`, mutation-checked.

2.8 consistency (`losses.py:755–808`)

```
P̂^start_bqℓ = 1 − Π_{c ∈ valid, s_c = ℓ} (1 − σ(p_bqc))
L_cons      = 0.5 · [ (Σ_{k_s} (P̂^start − σ(s))²) / |k_s| + end ]
```

- `k = (some candidate reaches ℓ) ∧ keep`.
- The target `σ(marginal)` is **not detached**, so the loss pulls on both the pair logits and the marginal logits.
- Weight `0.1 · min(t/2000, 1)`, a warmup from 0 (`trainer.py:1450–1460`).

2.9 abstention and count (`losses.py:811–859`)

- **Abstention:** L_null = (Σ_q BCE(n_bq, 1[q has no gold])) / #{valid q} Weight 0.2. At decode it gates entity and extractive queries only; `_decode_records` (events) never reads it.
- **Count:** L_count = mean_q (exp(r_bq) − y_bq · r_bq) Poisson NLL with `full=False`, so the value **can be negative** (−0.0375 on the traced record). Weight 0.2.

2.10 record loss, natural mode = events (`records.py` , `model.py`)

Object loss: identically 0 in `natural` mode (`records.py`).

Field, junction-column, and trigger-coref loss (`eb22g` vs `eb18`): - **Anchor seeding & coreferent ownership (`record_coreferent_ownership: true`):** - The gold trigger alternatives in `values[0]` are resolved against the model's anchor candidates. - If none is proposed or injected, the record trains nothing (`_note_anchor_gate`). - Under `record_coreferent_ownership: true` (`eb21` – `eb22g`), **every** resolved coreferent trigger mention `t_m ∈ G` seeds a gold instance row that owns the record's argument targets, with the record's role loss weighted by `1 / |G|` (in `eb18` , only `cols[0]` was seeded). - **Anchor self-row exclusion (`record_drop_anchor_field_loss: true` in `eb22f` / `eb22g`):** - `field_idx = 0` is the trigger anchor that seeded the instance. In `eb22f` / `eb22g` , it is dropped from the field loss (`record_drop_anchor_field_loss: true`), so `record_field_loss` averages strictly over argument roles `f >= 1` . - **Role assignment logits (`record_link_mode: junction` in `eb19` – `eb22g`):** s_ijf = P_ij + R_fj + <(U t_i) ⊙ (V a_j) ⊙ σ(G q_f), 1> / sqrt(d) + w_f · e_geom(i, j) - **Role list loss (`_list_field_bce` with `record_role_hard_negatives: 8`):** - **Empty gold role (`len(ft.values) == 0`):** averages BCE(`target = 0`) over the `K = 8` highest-scoring candidate columns. - **Non-empty gold role (`len(ft.values) > 0`) with `gold_cols == []` (not in candidate pool): withheld** (`0.0` loss, excluded from `fields_counted` ; in `eb21` – `eb22e` , this was erroneously trained as empty `target = 0`). - **Non-empty gold role with `gold_cols != []` :** any candidate overlapping an unresolved gold filler is masked out (`_unresolved_overlap_cols`), the top `K = 8` remaining non-gold columns are selected as hard negatives, and `_list_field_bce` **sums the separate positive and negative means**: ℓ_{r,f} = mean_{c ∈ gold_cols} BCE(s_c, 1) + mean_{c ∈ neg_cols} BCE(s_c, 0) (fixing the `(1 + K)` denominator in `eb21` – `eb22e` that pinned the equilibrium at `P = 1 / (1 + K) = 0.111`). - **Junction contrastive column loss (`record_link_column_weight: 0.3` in `eb19` – `eb22g`):** - For every gold argument column `j` of role `f` , a listwise softmax over triggers in column `j` maximizes the probability mass on the gold trigger(s) `G_j` against competing triggers: ℓ_col = −log (Σ_{i ∈ G_j} exp(s_ijf) / Σ_{i ∈ all_triggers} exp(s_ijf)) - **Trigger × trigger coreference loss (`record_coref_link: true` , `record_coref_link_weight: 0.3` , `coref_link_distance: mlp` width 64 in `eb22d` – `eb22g`):** - `TriggerLink` scores trigger instance pairs `(i, j)` via `<W t_i, W t_j> / sqrt(d) + MLP_64(g(i, j))` and computes split positive/hard-negative BCE (`record_coref_link_negatives: 4`) over same-event coreferent trigger pairs vs different-event same-type trigger pairs.

Batch aggregation (`eb22g`):

```
L_record = (Σ_{r, f ∈ counted} ℓ_{r,f}) / (# counted cells)
         + 0.3 · L_junction_col
         + 0.3 · L_trigger_coref
```

2.11 relation, entity coreference, and classification

- **Relation:** L_rel = (Σ_{mask} BCE(x_k, y_k)) / |mask| `y_k` is exact edge identity. Weight 1.0 (`model.py:1993–2075`).
- **Entity coreference (`entity_coref_link: true`, `entity_coref_link_weight: 0.3` in `eb21`–`eb22g`):** Pairwise bilinear + MLP-distance coreference loss over entity mentions (`entity_coref_loss`), weight 0.3 (`entity_coref_link_negatives: 4`).
- **Classification:** BCE over choices, divided by the label count, weight 1.0 (`model.py:1918–1969`).

2.12 silent guards (updated `2026–11–03`)

- `_finite_loss_term` (`nan_to_num`) zeroes non-finite head terms.
- **Fixed in `3c21b48` (`2026–11–03`):** `ValueError` and `IndexError` in record and relation losses are **no longer silently caught and skipped**; they now propagate immediately so shape or indexing bugs fail fast. Only `TargetCapacityError` is caught and handled according to `on_capacity_exceeded: truncate_with_warning`.

---

3. The compound equation (`eb22g` vs `eb18`, one micro-batch, optimizer step `t`)

```
L(t) = L_start + L_end + L_pair[split_pos_neg] + 0.5 · L_inside
     + 0.2 · max(1 − t/20000, 0) · L_iou
     + 0.3 · L_rerank + 0.3 · L_prop[identity]
     + 0.1 · min(t/2000, 1) · L_cons + 0.2 · L_null + 0.2 · L_count
     + L_cls + 1.0 · L_record[split_bce + withhold + junction + trigger_coref]
     + 0.3 · L_entity_coref + 1.0 · L_rel
```

- Every term passes through `nan_to_num`.
- `p_inj(t) = 1.0` for the first 15% of planned steps, then linear to `0.25`. It enters through:
- the candidate positives (pair, rerank, soft-IoU),
- the record anchor gate (`_note_anchor_gate`; `L_prop` uses `proposal_gold: identity` so naturally proposed gold is never flipped to negative as `p_inj` decays).

The update:

```
θ ← AdamW(θ, clip_{1.0}( Σ_{m=1..grad_accum} ∇_θ (L_m(t) / grad_accum) ))
```

with encoder LR 2e-5, task LR 5e-4, and cosine_restarts.

Reading gradient shares under AdamW. AdamW divides each coordinate by its own running RMS. - A module touched by one term alone steps at full size, whatever its global share. The record decoder gets gradient only from `record_field`. - Shares are therefore meaningful **within a parameter group**, where the terms compete for the same coordinates. - Global-norm clipping and accumulation do not change shares: clipping scales uniformly, and accumulation is linear.

Per-group shares, one training-shaped batch (synthetic_events_capped, sentence_rex, docee_zh, docee; step-0 head schedules; each group sums to 1.0000; residual 0.0):

group	rerank	pair	proposal	count	abstention	soft_iou	inside	start	end
boundary_head	0.333	0.319	0.257	0.005	0.0001	0.083	0.001	0.0003	0.0003
encoder	0.298	0.178	0.195	0.212	0.041	0.039	0.016	0.0045	0.0040

Gradient shares over 8 batches (`pooled`, eb18's loss)

Setup: `measure_gradient_shares.py --batches 8`. Each batch is 4 chunks from the natural mix. The run uses step-0 head schedules (p_inj 1.0, soft-IoU 1.0, consistency 1.0). Residual is 0.0 on every batch, and every group sums to 1.0000.

Shares are mean (min, max). Single-term groups (classifier, record_decoder, relations) are 1.0 by construction.

term	boundary_head	encoder
rerank_listwise	0.297 (0.126, 0.577)	0.314 (0.081, 0.430)
proposal	0.260 (0.075, 0.506)	0.248 (0.093, 0.345)
pair	0.263 (-0.136, 0.605)	0.147 (0.107, 0.178)
soft_iou	0.137 (-0.063, 0.616)	0.045 (0.027, 0.092)
count	0.038	0.109 (0.015, 0.294)
classification	0	0.113 (0.000, 0.480)
abstention	0.000	0.009
relation	0	0.008
record_field	0.007	0.002
inside	0.0007	0.005
start	**0.0005**	**0.0023**
end	**0.0004**	**0.0020**
consistency	-0.0002	-0.0001

Reading it: - Three listwise and pairwise candidate terms (rerank, proposal, pair) set about 0.82 of the boundary head's direction and 0.71 of the encoder's. - The start/end boundary terms set under 0.1% of the boundary head and about 0.4% of the encoder. That is the budget fix #2 (`absent_reduction`) acts on. -

Pair and soft-IoU go **negative** on some batches. On those batches they opposed the rest of the update in the boundary head, which fits the pair/soft-IoU target conflict in 2.5.

`absent_reduction: separate` (same 8 batches, residual 0.0 on all) raises the boundary terms by about 1.5×:

group	term	pooled	separate
boundary_head	start	0.0005	0.0008
boundary_head	end	0.0004	0.0006
encoder	start	0.0023	0.0034
encoder	end	0.0020	0.0031

Every other term is unchanged to the third decimal. Fix #2 does what its equation says, on a budget too small to be likely to move a score: under 0.4% of the encoder's direction.

Late-training state: the contradicted term dominates

Setup: p_inj 0.25, soft-IoU 0, consistency 1. This is where eb18 spent most of its steps. Both modes ran on the same 8 seeded batches, with residual 0.0 on all of them. Values are mean (min, max).

term	boundary_head, injected (eb18)	boundary_head, identity (fix)	encoder, injected (eb18)	encoder, identity (fix)
proposal	**0.590** (0.372, 0.828)	0.304 (0.121, 0.440)	**0.491** (0.272, 0.688)	0.304 (0.207, 0.402)
rerank_listwise	0.223	0.367	0.243	0.350
pair	0.172	0.291	0.105	0.131
count	0.013	0.034	0.087	0.108
record_field	0.0008	0.0056	0.0006	0.0014
start / end	0.0003 / 0.0003	0.0007 / 0.0007	0.0018 / 0.0017	0.0023 / 0.0022

What this shows: - In eb18's late state, the proposal loss set **59% of the boundary head's update direction and 49% of the encoder's**. At p_inj 0.25 that term labels most naturally found gold as negatives (14 of 16 in the 2.7 trace). - Early (p_inj 1.0) it was about a quarter of the update. The contradiction grows into the dominant term exactly when injection anneals. - Under `proposal_gold: identity` the proposal share returns to 0.30, close to its early level, and rerank and pair regain theirs.

This is mechanism, not outcome: the Phase 2 fast A/B (catalog, `p2fast`) measures whether it moves scores.

---

4. eb19 candidates, as deltas to section 3

Each candidate is opt-in and none is adopted yet. 1. **`absent_reduction: separate`** (built, `df162a0`) - Replaces π_P·Ł_P + π_A·Ł_A in L_start/L_end with s·Ł_P + λ_A·Ł_A. - Touches start/end only. These carried 0.0003 (boundary_head) and about 0.004 (encoder) of their groups on one batch, so the expected effect is small. 2. **Proposal gold by span identity** (built, `proposal_gold: identity`) - In L**rop, G^prop_bq = {c : z_bqc > 0.5}. -** is removes the dependence of L_prop on p_inj(t) for gold that was proposed naturally. 3. **Per-corpus soft target for absent queries** (idea) - In Ł_A and the absent rows of L_pair, set y = ε_c instead of 0. - ε_c would be ranked from the adjudication below.

---

5. Findings from tracing

1. **Gold reaches 100% of training candidates only for the first 15% of steps.** - p_inj then anneals to 0.25. - An earlier statement that "eb18 already reaches 100% of gold triggers in training" was true only for that window. It is corrected here.
2. **`negative_query_ratio` sampling is inert under the global reduction.** All absent queries contribute to every term that sees them.
3. **The trace's S6 weights are step-0 values.** - In training, soft-IoU decays to 0 by 20,000 steps and consistency warms up over 2,000. - A warm start resets t, so it re-enables soft-IoU and restarts injection at 1.0.
4. **The proposal loss trains against naturally found gold whenever p_inj < 1** (2.7, probe table).
5. **Absent-event negatives that really occur** (Haiku 4.5 judge, 50 random injections per corpus): - wikievents 34%, maven 28%, cmnee 20%, cc_news_haiku45 14%, synthetic 12%, cc_news_events 10%, casie 6%, rams 6%, duee 0%. - The judge over-calls fine-grained taxonomies, so these are upper bounds. - The lexicon detector's recall against the judge is mostly 0.00-0.33. - Catalog row: 2026-10-02 "Absent-EVENT negatives adjudicated".
6. **Dilution of the boundary loss:** π_P 0.862 → 0.713 with negatives on. The worst corpora are rams_merged 0.372 and cc_news_events 0.420. Catalog row: 2026-10-02 "Absent queries dilute the boundary loss".
7. **Gradient magnitude is not loss magnitude.** - On one record, proposal and rerank supplied 0.69 of the gradient direction from 56% of the loss (0.453 of 0.807). - start + end supplied 0.016 of the direction from 5.6% of the loss.

6. Status of the 2026-10-02 open questions (as of **`eb22g`**, 2026-11-05)

- **Injection schedule × record anchor gate (`_note_anchor_gate`):**
- **Measured (`probe_role_candidates.py`, paired collate on `eb22e:checkpoint-epoch-1`, 2026-11-03):** Under Epoch-1 scheduled injection (`0.9559`), `97.1%` of `3,568` gold role fillers are in the candidate pool (`0.6%` trigger not proposed, `2.3%` span not in pool). With injection off (`0.0`), `38.3%` are candidates, `28.8%` are not in the pool, and `32.9%` lose their trigger — traced to

`candidate_pair_loss` absent-query dilution and fixed in `eb22g` (`pair_split_reduction: true`, `hard_negative_keep_all_when_absent: false`).

- **Record field normalisation (scalar trigger NLL vs list role BCE):**
- **Resolved (`eb22f` / `eb22g`):** (1) `record_drop_anchor_field_loss: true` drops the scalar trigger anchor self-row (`field_idx = 0`) from `record_field_loss` in `natural` mode; (2) `record_role_hard_negatives: 8` selects at most 8 hard negatives per role; (3) `_list_field_bce` splits the positive and hard-negative means (`pos_loss.mean() + neg_loss.mean()`); and (4) unresolved non-empty gold roles (`gold_cols == []`) are withheld (`0.0` loss) instead of trained as empty.
- **Soft-IoU conflict:** Same logit still receives target 0 from `pair` and fractional IoU from `soft_iou` on overlapping hard negatives for the first 20,000 steps (`σ_iou(t) = max(1 − t/20000, 0)`), after which `soft_iou` switches off.
- **Consistency's undetached target:** Retained unchanged (`weight = 0.1 · min(t/2000, 1)`).
- **Silent exception guards in record/relation losses:**
- **Resolved (`3c21b48`, 2026-11-03):** `ValueError` and `IndexError` now propagate immediately; only `TargetCapacityError` is caught and handled per `on_capacity_exceeded`.
- **Abstention unused for events at decode:** Retained as designed; event instance admission is gated by the calibrated trigger/record threshold (`threshold_sweep.json`) and `TriggerLink` coreference calibration (`coref_calibration.py`).